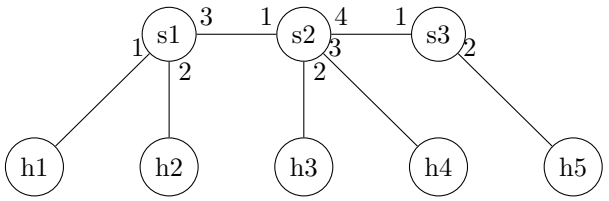
Piattaforme Software per la Rete (Mod. 1) – 13/07/2016

Esercizio 1

Si consideri la rete in figura. I nodi h1,..., hn contengono tabelle ARP prepopolate per tutta la rete. Gli indirizzi MAC e IP dei nodi h1,..., hn sono MAC1,..., MACn e IP1,..., IPn. Sui nodi h1,..., hn sono in esecuzione processi in ascolto su tutte le porte TCP e UDP di interesse. Il controllore Ryu è collegato ai nodi s1,..., sm ed è configurato come indicato nel Programma 1.



(a) Scrivere il contenuto delle tabelle openflow dopo l'accensione di tutti i nodi.

Soluzione:

s1

Priority	Match	Action	pkt cnt
0	*	output controller	0
1	eth_src = MAC1 eth_type = 0x800	output flood	0

s2

Priority	Match	Action	pkt cnt
0	*	output controller	0
1	eth_dst = MAC5 eth_type = 0x800 ip_proto = 17	set eth_dst=MAC4 set ip_dst=IP4 ouput 3	0

s3

Priority	Match	Action	pkt cnt
0	*	output controller	0

L'host h1, porta 123 invia un datagramma UDP a h5, porta 123.

(b) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s1.

Soluzione:
s1 → h2, s2: MAC1, MAC5, IPv4, IP1, IP5, TCP, 123, 123

(c) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2 e s3.

Soluzione: s2 → h4: MAC1, MAC4, IP1, IP4, IPv4, 123, 123

L'host h3, porta 5432 invia una richiesta di apertura di connessione a h2, porta 80.

Programma 1 (1/2) – Switch Features

```
# intestazione omessa
@set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
def switch_features_handler(self, ev):
    datapath = ev.msg.datapath
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser

    match = parser.OFPMatch()
    actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
                                      ofproto.OFPCML_NO_BUFFER)]
    inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
                                         actions)]
    mod = parser.OFPFlowMod(datapath=datapath, priority=0,
                            match=match, instructions=inst)
    datapath.send_msg(mod)

    if (datapath.id == 1):
        match = parser.OFPMatch(
            eth_src=MAC1,
            eth_type=0x0800)    # IPv4
        actions = [ parser.OFPActionOutput(ofproto.OFPP_FLOOD) ]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
                                             actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=1,
                                match=match, instructions=inst)
        datapath.send_msg(mod)

    if (datapath.id == 2):
        match = parser.OFPMatch(
            eth_dst=MAC5,
            eth_type=0x0800,    # IPv4
            ip_proto=17)        # UDP
        actions = [
            parser.OFPActionSetField(eth_dst=MAC4),
            parser.OFPActionSetField(ip_dst=IP4),
            parser.OFPActionOutput(3)
            ]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
                                             actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=1,
                                match=match, instructions=inst)
        datapath.send_msg(mod)
```

Programma 1 (2/2) – Packet In

```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def packet_in_handler(self, ev):
    msg = ev.msg; datapath = msg.datapath
    ofproto = datapath.ofproto; parser = datapath.ofproto_parser
    in_port = msg.match['in_port']
    pkt = packet.Packet(msg.data)
    pkt_eth = pkt.get_protocol(eth.eth)
    pkt_ipv4 = pkt.get_protocol(ipv4.ipv4)
    pkt_udp = pkt.get_protocol(udp.udp)
    pkt_tcp = pkt.get_protocol(tcp.tcp)

    if (datapath.id == 2) and (pkt_tcp is not None):
        actions = [parser.OFPActionOutput(ofproto.OFPP_FLOOD)]
        out = parser.OFPPacketOut(datapath=datapath, in_port=in_port,
                                  actions=actions, data=data)
        datapath.send_msg(out)

        match = parser.OFPMatch(
            eth_dst=pkt_eth.src,
            eth_type=0x0800,    # IPv4
            ip_proto=6)         # TCP
        actions = [ parser.OFPActionOutput(in_port) ]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
                                             actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=1,
                                match=match, instructions=inst)
        datapath.send_msg(mod)

        return
```

(d) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

Soluzione:
s2 → C: packet_in(MAC3,MAC2, IPv4, IP3, IP2, TCP, 5432, 80)
s2 → s1,s3,h4: MAC3,MAC2, IPv4, IP3, IP2, TCP, 5432, 80

Il server risponde all'apertura di connessione

(e) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

Soluzione:
Il pacchetto SYN è scartato in s1 e non arriva al server.

(f) Scrivere il contenuto delle tabelle openflow.

Soluzione:

s1

Priority	Match	Action	pkt cnt
0	*	output controller	0
1	eth_src = MAC1 eth_type = 0x800	output flood	1

s2

Priority	Match	Action	pkt cnt
0	*	output controller	0
1	eth_dst = MAC5 eth_type = 0x800 ip_proto = 17	set eth_dst=MAC4 set ip_dst=IP4 output 3	1
1	eth_dst = MAC3 eth_type = 0x800 ip_proto = 6	output 2	0

s3

Priority	Match	Action	pkt cnt
0	*	output controller	0

Esercizio 2

Un sistema di backtracking usa filtri di Bloom per tenere traccia dei pacchetti transitati da un router. Il router riceve pacchetti alla velocità di 10 Mbit/s. Si assuma lunghezza minima dei pacchetti pari a 50 byte.

Per memorizzare i pacchetti osservati si calcolano 3 funzioni hash di l bit sull'intero pacchetto.

(a) Si assumano funzioni hash ideali di $l = 16$ bit e un intervallo di osservazione $T = 600\,\text{ms}$. Calcolare la probabilità di falso positivo.

Soluzione:
Sia n il numero di inserimenti, k il numero di hash, m la lunghezza dell'array.

$$
\begin{aligned}
n &= \frac{10 \cdot 10^6\,\text{bit/s}}{50 \times 8} 0{,}6\,\text{s} \\
k &= 3 \\
m &= 2^{16} \\
p &= \left(1 - \exp\left(-\frac{kn}{m}\right)\right)^k = 0.12
\end{aligned}
$$

(b) Anziché i singoli pacchetti si vuole effettuare il backtracking di connessioni TCP. Come si deve modificare l'algoritmo? C'è un impatto sulla probabilità di falso positivo?

Soluzione:
Si effettua l'hash della quintupla che definisce il flusso.
Poiché il numero di connessioni è minore del numero di pacchetti, ci saranno meno inserimenti e meno falsi positivi.

Si considerino le seguenti funzioni hash con $l = 3$:

$$
\begin{aligned}
h_1(x) &= x + 1 \bmod 2^3 \\
h_2(x) &= 3x + 2 \bmod 2^3 \\
h_3(x) &= 7x + 3 \bmod 2^3
\end{aligned}
$$

(c) Si inseriscano nel filtro i valori $x = 0$, $x = 2$.

Soluzione:
Per $x = 0$ sono posti a 1 i bit 1, 2, 3.
Per $x = 2$ sono posti a 1 i bit 0, 1, 3.
In totale sono posti a 1 i bit 0, 1, 2, 3

(d) Verificare se sono presenti nel filtro gli elementi $x = 5$, $x = 10$. Quali di questi sono falsi positivi?

Soluzione:
Per $x = 0$ si verificano i bit 1, 6. Non presente.
Per $x = 10$ si verificano i bit 0, 1, 3. Presente (falso positivo).

Esercizio 3

Si consideri una interfaccia di uscita gestita da uno scheduler Deficit Round Robin. La velocità di trasmissione è 8 Mbit/s. Al tempo $t = 0$ sono presenti i seguenti con la seguenti lunghezze in byte, ordinati dal più recente al più vecchio:

Coda 1 150, 75, 150

Coda 2 50, 150, 225

Coda 3 –

Il quanto è 100 byte per tutte le code. Al tempo $T = 100\,\mu s$ si presenta alla coda 3 un pacchetto da 50 byte. Al tempo $T = 200\,\mu s$ si presenta alla coda 3 un pacchetto da 100 byte.

(a) Simulare l'algoritmo DRR, indicando ad ogni tempo quali pacchetti concludono la trasmissione.

Soluzione:

Iter	Coda	Deficit	Lunghezza Pacchetto (byte)	Fine Trasmissione (µs)
1	1	100	–	–
1	2	100	–	–
1	3	–	–	–
2	1	50	150	150
2	2	200	–	–
2	3	50	50	200
3	1	75	75	275
3	2	75	225	500
3	3	–	100	600
4	1	–	150	750
4	2	25	150	900
4	3	–	–	–
5	1	–	–	–
5	2	–	50	950

(b) Considerando dimensione massima dei pacchetti pari a 500 byte, calcolare il tempo massimo che un pacchetto che si presenta alla coda 3 trovandola vuota deve aspettare prima di essere trasmesso.

Soluzione:

Nel caso pessimo abbiamo: un pacchetto in trasmissione per la coda 3 e ciascuna delle code 1 e 2 hanno un deficit accumulato di 499 ed hanno due pacchetti in coda: uno da 500 byte e uno da 499.

Bisogna quindi aspettare $500 + 500 + 500 + 499 + 499 = 2{,}498\,ms$.

(c) Simulare la stessa interfaccia della prima domanda, ma con uno scheduler a priorità semplice ($1 > 2 > 3$). Dire quanto tempo aspettano in coda i due pacchetti della coda 3 nei casi di scheduler DRR e di scheduler a priorità.

Soluzione:

Lunghezza Pacchetto (byte)	Fine Trasmissione (µs)
150	150
75	225
150	375
225	600
150	750
50	800
50	850
100	950

Nel caso DRR il primo pacchetto aspetta $50\,\mu s$, il secondo aspetta $300\,\mu s$.

Nel caso a priorità il primo pacchetto aspetta $700\,\mu s$, il secondo aspetta $650\,\mu s$.